S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR.

(An Autonomous Institute, Affiliated to RTMNU, Nagpur)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

"To become a center for quality education in the field of computer science & engineering and to create competent professionals."

TAE-1 Report (2026-27)

Advanced Data Structures (PCCCE102T)

Name of the Student: TUSHAR PRADYUMNA BISANE

USN No : CE26021

MTech-Semester : Ist Section: CSE

Title of Problem: _____

Date of Submission: 05/11/2026

Marks: -

PROBLEM STATEMENT	MARKS	IMPLEMENTATION 5M	VIVA 5M	TOTAL
PROBLEM STATEMENT 1	10M			
PROBLEM STATEMENT 2	10M			
PROBLEM STATEMENT 3	10M			
PROBLEM STATEMENT 4	10M			
	40M	**NOTE: 40 marks will be converted into 10 marks based on averaging**		

Sign of Student **Sign of Subject In charge**

PROBLEM STATEMENT 1

Unit 1

A Symbol Table is an important data structure used in compiler design to store and manage information about identifiers such as variable names, function names and their associated attributes.

The objective of this problem is to implement a Symbol Table using the Open Addressing technique with Linear Probing to resolve collisions.

A fixed-size hash table of size 7 is used to store integer keys. A hash function calculates the initial index for each key. If a collision occurs, the next consecutive position is checked until an empty position is found.

Operations to be implemented:

1. Insert (Key)
2. Search (Key)
3. Display (Hash Table)

Hash Function:

$$h(key) = key \bmod 7$$

Linear Probing Formula:

$$h(key, i) = (h(key) + i) \bmod 7$$

Where:

- key = Element to be inserted.
- i = Number of attempts or probe count.
- 7 = Size of the hash table.

Example:
Insert the keys 10, 17 and 24.

Key	Hash calculation	Position
10	10 % 7 = 3	3
17	17 % 7 = 3	4
24	24 % 7 = 3	5

The keys 17 and 24 experience collisions at index 3. Linear probing places them at the next available positions, 4 and 5.

Expected Output:
Index 0: Empty
Index 1: Empty
Index 2: Empty
Index 3: 10
Index 4: 17
Index 5: 24
Index 6: Empty

Theory :

Algorithm :

Flowchart:

Solution in C++

Discussion on Solution:

The program implements a Symbol Table using the Open Addressing technique with Linear Probing.

A fixed-size array of 7 elements is used to store integer keys. The hash function calculates the initial index using the modulo operation.

When a key is inserted, its hash index is calculated. If the position is empty, the key is inserted directly. If the position is already occupied, a collision occurs and the program checks the next consecutive position.

The same probing technique is used during searching to locate the required key.

The display operation traverses the complete hash table and prints the stored keys along with empty positions.

The program also handles duplicate keys and table overflow conditions.

Thus, Linear Probing resolves collisions by searching for the next available position within the same hash table.

Screen Shots of Program :

main.cpp

```cpp

#include <iostream>
using namespace std;

#define SIZE 7

int hashTable[SIZE];
bool occupied[SIZE] = {false};

// Function to insert a key
void insertKey(int key)
{
    int index = key % SIZE;

    for (int i = 0; i < SIZE; i++)
    {
        int pos = (index + i) % SIZE;

        if (occupied[pos] && hashTable[pos] == key)
        {
            cout << "Key Already Exists!" << endl;
            return;
        }

        if (!occupied[pos])
        {
            if (i > 0)
                cout << "Collision detected! ";
```

main.cpp

```cpp
            hashTable[pos] = key;
            occupied[pos] = true;

            cout << "Inserted at index "
                 << pos << endl;
            return;
        }
    }

    cout << "Table Full (Overflow)" << endl;
}

// Function to search a key
void searchKey(int key)
{
    int index = key % SIZE;

    for (int i = 0; i < SIZE; i++)
    {
        int pos = (index + i) % SIZE;

        if (!occupied[pos])
        {
            cout << "Key Not Found" << endl;
            return;
        }

        if (hashTable[pos] == key)
```

main.cpp

```cpp
        {
            cout << "Key Found at index "
                 << pos << endl;
            return;
        }
    }

    cout << "Key Not Found" << endl;
}

// Function to display the hash table
void displayTable()
{
    cout << "\n--- HASH TABLE ---" << endl;

    for (int i = 0; i < SIZE; i++)
    {
        cout << "Index " << i << ": ";

        if (occupied[i])
            cout << hashTable[i];
        else
            cout << "Empty";

        cout << endl;
    }
}

```

main.cpp

```
76
77          if (occupied[i])
78              cout << hashTable[i];
79          else
80              cout << "Empty";
81
82          cout << endl;
83      }
84  }
85
86  // Main function
87  int main()
88  {
89      int choice, key;
90
91      do
92      {
93          cout << "\n--- SYMBOL TABLE ---" << endl;
94          cout << "1. Insert" << endl;
95          cout << "2. Search" << endl;
96          cout << "3. Display" << endl;
97          cout << "0. Exit" << endl;
98
99          cout << "Enter Choice: ";
100         cin >> choice;
101
102         switch (choice)
103         {
```

main.cpp

```
105             cout << "Enter Key: ";
106             cin >> key;
107             insertKey(key);
108             break;
109
110         case 2:
111             cout << "Enter Key: ";
112             cin >> key;
113             searchKey(key);
114             break;
115
116         case 3:
117             displayTable();
118             break;
119
120         case 0:
121             cout << "Program Exited." << endl;
122             break;
123
124         default:
125             cout << "Invalid Choice!" << endl;
126         }
127
128     } while (choice != 0);
129
130     return 0;
131 }
```

Result with Various Test Cases:

```
--- SYMBOL TABLE ---
1. Insert
2. Search
3. Display
0. Exit
Enter Choice: 1
Enter Key: 10
Inserted at index 3

--- SYMBOL TABLE ---
1. Insert
2. Search
3. Display
0. Exit
Enter Choice: 1
Enter Key: 17
Collision detected! Inserted at index 4

--- SYMBOL TABLE ---
1. Insert
2. Search
3. Display
0. Exit
Enter Choice: 1
Enter Key: 24
Collision detected! Inserted at index 5
```

```
--- SYMBOL TABLE ---
1. Insert
2. Search
3. Display
0. Exit
Enter Choice: 3

--- HASH TABLE ---
Index 0: Empty
Index 1: Empty
Index 2: Empty
Index 3: 10
Index 4: 17
Index 5: 24
Index 6: Empty
```

Time Complexity and Discussion :

Time complexity describes the amount of time required by an algorithm to execute as the input size increases.

In this problem, the hash table uses Linear Probing to perform insertion, searching and display operations.

Let n represent the number of positions in the hash table.

Time Complexity Table

Operation	Best Case	Average Case	Worst Case
Insertion	O(1)	O(1)	O(n)
Searching	O(1)	O(1)	O(n)
Display	O(n)	O(n)	O(n)

Explanation:

1. Insertion - O(1) Average Case

When the calculated hash position is empty, the key is inserted directly. If collisions occur, additional positions are checked.

In the worst case, the entire table must be checked, resulting in O(n) time complexity.

2. Searching - O(1) Average Case

The search operation calculates the hash index and checks the corresponding position.

If collisions occur, Linear Probing checks consecutive positions until the key is found or an empty position is encountered.

In the worst case, all positions may need to be checked, resulting in O(n).

3. Display - O(n)

The display operation traverses every position of the hash table and prints its contents.

Therefore, its time complexity is O(n).

Advantages

1. Simple implementation.
2. Efficient insertion and searching when the table has sufficient empty space.
3. No additional linked-list memory required.
4. Good cache performance.

Limitations

1. Primary clustering can occur.
2. Performance decreases when the table becomes full.

PROBLEM STATEMENT 2

Unit 2

Implement a Skip List to store train ticket PNR numbers. The Skip List should support fast searching and insertion operations.

Use a random coin flip to determine the level of each newly inserted node.

Operations to be implemented:

1. Insert (PNR)
2. Search (PNR)
3. Delete (PNR)
4. Display (Level by Level)

Theory :

Algorithm :

Flowchart:

Solution in C++

Discussion on Solution:

The program uses a Skip List to maintain train ticket PNR numbers in sorted order.

Random levels are generated using a coin-flipping technique. Each node contains a PNR number and multiple forward pointers.

The insertion operation places a PNR at its correct sorted position. Searching starts from the highest level and moves downward. Deletion removes the selected PNR and updates the corresponding pointers.

The display operation prints the Skip List level by level.

Screen Shots of Program :




```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

const int MAX_LEVEL = 5;
const float P = 0.5;

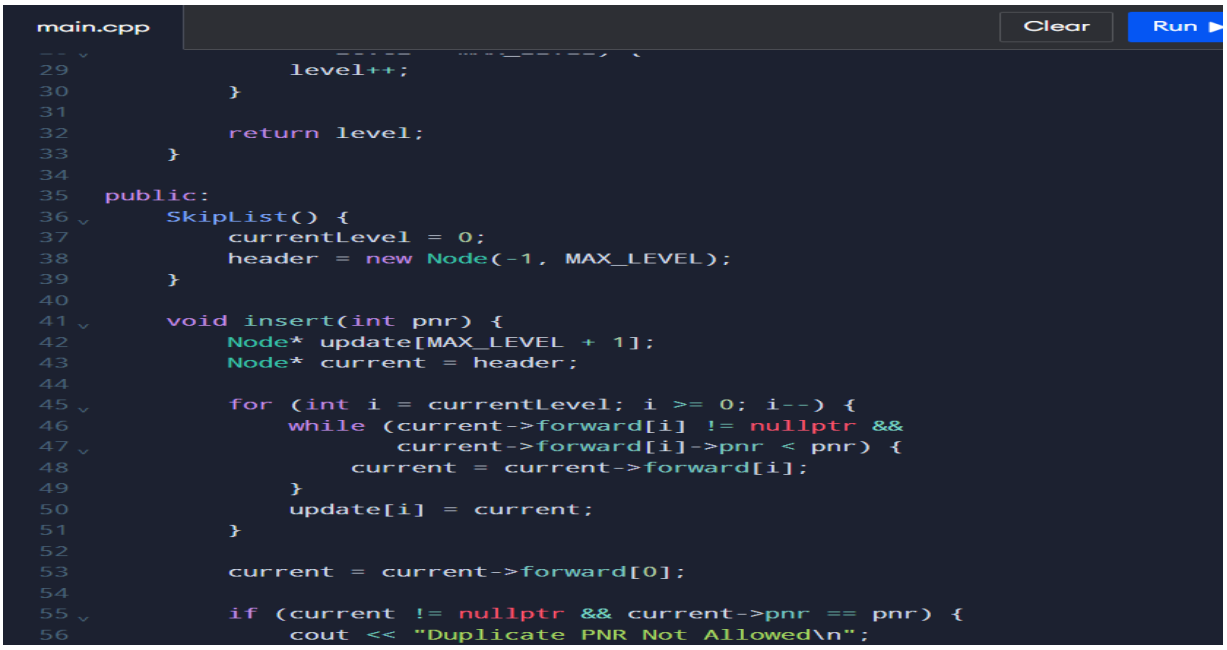
struct Node {
    int pnr;
    Node* forward[MAX_LEVEL + 1];

    Node(int value, int level) {
        pnr = value;
        for (int i = 0; i <= MAX_LEVEL; i++)
            forward[i] = nullptr;
    }
};

class SkipList {
    Node* header;
    int currentLevel;

    int randomLevel() {
        int level = 0;

        while ((float)rand() / RAND_MAX < P &&
               level < MAX_LEVEL) {
```




```cpp
            level++;
        }

        return level;
    }

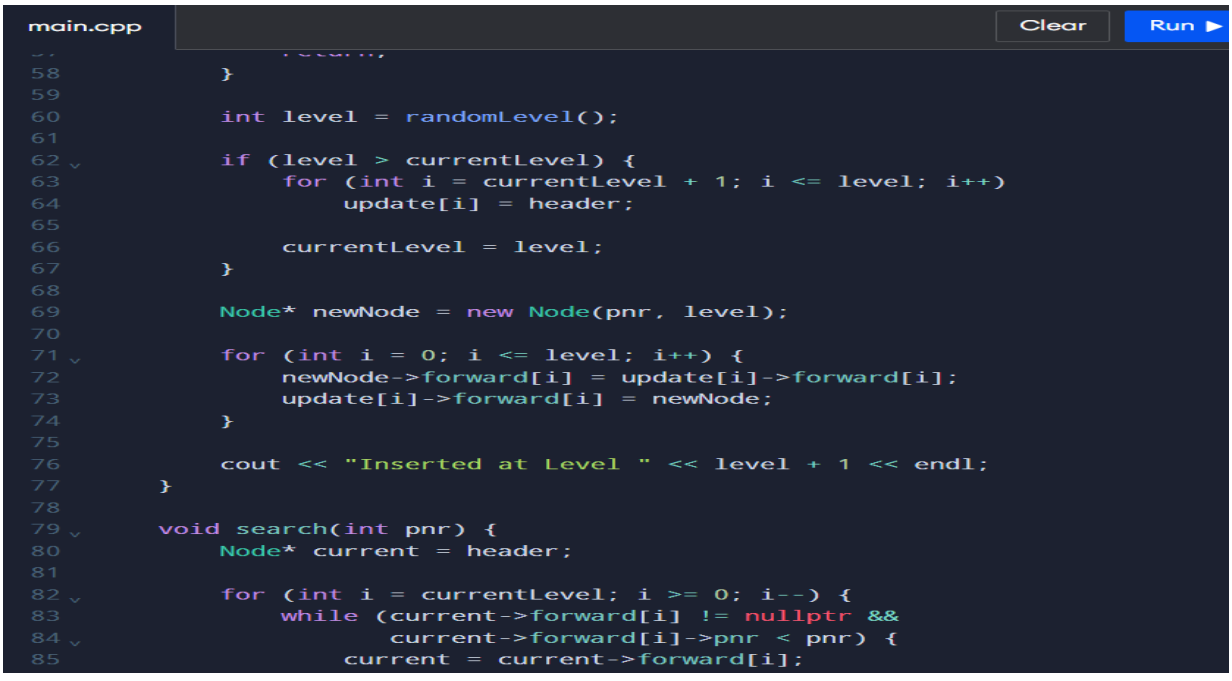
public:
    SkipList() {
        currentLevel = 0;
        header = new Node(-1, MAX_LEVEL);
    }

    void insert(int pnr) {
        Node* update[MAX_LEVEL + 1];
        Node* current = header;

        for (int i = currentLevel; i >= 0; i--) {
            while (current->forward[i] != nullptr &&
                   current->forward[i]->pnr < pnr) {
                current = current->forward[i];
            }
            update[i] = current;
        }

        current = current->forward[0];

        if (current != nullptr && current->pnr == pnr) {
            cout << "Duplicate PNR Not Allowed\n";
```




```cpp
        }

        int level = randomLevel();

        if (level > currentLevel) {
            for (int i = currentLevel + 1; i <= level; i++)
                update[i] = header;

            currentLevel = level;
        }

        Node* newNode = new Node(pnr, level);

        for (int i = 0; i <= level; i++) {
            newNode->forward[i] = update[i]->forward[i];
            update[i]->forward[i] = newNode;
        }

        cout << "Inserted at Level " << level + 1 << endl;
    }

    void search(int pnr) {
        Node* current = header;

        for (int i = currentLevel; i >= 0; i--) {
            while (current->forward[i] != nullptr &&
                   current->forward[i]->pnr < pnr) {
                current = current->forward[i];
```

main.cpp Clear Run ▶

```cpp
    void deletePNR(int pnr) {
        Node* update[MAX_LEVEL + 1];
        Node* current = header;

        for (int i = currentLevel; i >= 0; i--) {
            while (current->forward[i] != nullptr &&
                   current->forward[i]->pnr < pnr) {
                current = current->forward[i];
            }
            update[i] = current;
        }

        current = current->forward[0];

        if (current == nullptr || current->pnr != pnr) {
            cout << "PNR Not Found\n";
            return;
        }

        for (int i = 0; i <= currentLevel; i++) {
            if (update[i]->forward[i] != current)
                break;

            update[i]->forward[i] = current->forward[i];
        }

        delete current;

        while (currentLevel > 0 &&
```

main.cpp Clear Run ▶

```cpp

            cout << "Level " << i + 1 << ": ";

            while (current != nullptr) {
                cout << current->pnr << " -> ";
                current = current->forward[i];
            }

            cout << "NULL\n";
        }
    }
};

int main() {
    srand(time(0));

    SkipList list;
    int choice, pnr;

    do {
        cout << "\n--- TRAIN TICKET SKIP LIST ---\n";
        cout << "1. Insert PNR\n";
        cout << "2. Search PNR\n";
        cout << "3. Delete PNR\n";
        cout << "4. Display Skip List\n";
        cout << "0. Exit\n";

        cout << "Enter Choice: ";
```

main.cpp Clear Run ▶

```cpp
            case 2:
                cout << "Enter PNR: ";
                cin >> pnr;
                list.search(pnr);
                break;

            case 3:
                cout << "Enter PNR: ";
                cin >> pnr;
                list.deletePNR(pnr);
                break;

            case 4:
                list.display();
                break;

            case 0:
                cout << "Program Exited\n";
                break;

            default:
                cout << "Invalid Choice\n";
        }

    } while (choice != 0);

    return 0;
}
```

Result with Various Test Cases:

```
--- TRAIN TICKET SKIP LIST ---
1. Insert PNR
2. Search PNR
3. Delete PNR
4. Display Skip List
0. Exit
Enter Choice: 1
Enter PNR: 1001
Inserted at Level 4

--- TRAIN TICKET SKIP LIST ---
1. Insert PNR
2. Search PNR
3. Delete PNR
4. Display Skip List
0. Exit
Enter Choice: 2
Enter PNR: 1002
PNR Not Found

--- TRAIN TICKET SKIP LIST ---
1. Insert PNR
2. Search PNR
3. Delete PNR
4. Display Skip List
0. Exit
Enter Choice: 1
Enter PNR: 1001
```

```
4. Display Skip List
0. Exit
Enter Choice: 1
Enter PNR: 1001
Duplicate PNR Not Allowed

--- TRAIN TICKET SKIP LIST ---
1. Insert PNR
2. Search PNR
3. Delete PNR
4. Display Skip List
0. Exit
Enter Choice: 4
Level 4: 1001 -> NULL
Level 3: 1001 -> NULL
Level 2: 1001 -> NULL
Level 1: 1001 -> NULL

--- TRAIN TICKET SKIP LIST ---
1. Insert PNR
2. Search PNR
3. Delete PNR
4. Display Skip List
0. Exit
Enter Choice: 1
Enter PNR:

=== Session Ended Due To Inactivity ===
```

Time Complexity and Discussion :

Time complexity describes the amount of time required by an algorithm to execute as the input size increases.

In this problem, the hash table uses Linear Probing to perform insertion, searching and display operations.

Let n represent the number of positions in the hash table.

Time Complexity Table

Operation	Average Case	Worst Case
Insertion	O(log n)	O(n)
Searching	O(log n)	O(n)
Display	O(n)	O(n)
Deletion	O(log n)	O(n)

Explanation:

- Searching is efficient because higher levels allow the algorithm to skip multiple nodes.
- Insertion requires finding the correct position and updating pointers.
- Deletion requires searching for the node and adjusting pointers.
- Display visits all nodes at each displayed level.

Conclusion:

The Randomized Skip List for Train Ticket Booking was successfully implemented using C++.

The program demonstrates insertion, searching, deletion and display operations. Random level generation helps improve searching efficiency compared with a simple linked list.

PROBLEM STATEMENT 3

Unit 5

Implement a 1D Range Searching program using Binary Search Tree (BST).

Given range [a, b] find and display all points keys stored in the Binary Search Tree that lie within the specific range.

Operations to be implemented:

1. Insert (Key)
2. Range (a,b)

Theory :

Algorithm :

Flowchart:

Solution in C++

Discussion on Solution:

The program implements a Binary Search Tree for performing one-dimensional range searching.

Each node contains an integer key and two pointers representing the left and right child nodes.

The insertion operation places keys according to BST properties. The range searching operation checks whether a node falls within the specified lower and upper limits.

The algorithm avoids unnecessary traversal by checking the current node's value before visiting its subtrees.

The program also provides an inorder display operation to print the stored keys in ascending order.

Screen Shots of Program :

main.cpp Clear Run ▶

```cpp
#include <iostream>
using namespace std;

// Structure of BST node
struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int value) {
        data = value;
        left = nullptr;
        right = nullptr;
    }
};

// Insert a key into BST
Node* insert(Node* root, int key) {
    if (root == nullptr) {
        return new Node(key);
    }

    if (key < root->data) {
        root->left = insert(root->left, key);
    }
    else if (key > root->data) {
        root->right = insert(root->right, key);
    }
```

main.cpp Clear Run ▶

```cpp
    }

    return root;
}

// Perform range searching
void rangeSearch(Node* root, int a, int b,
                 bool &found) {
    if (root == nullptr)
        return;

    // Search left subtree
    if (root->data > a) {
        rangeSearch(root->left, a, b, found);
    }

    // Check current node
    if (root->data >= a && root->data <= b) {
        cout << root->data << " ";
        found = true;
    }

    // Search right subtree
    if (root->data < b) {
        rangeSearch(root->right, a, b, found);
    }
}
```

main.cpp Clear Run ▶

```cpp
    if (root == nullptr)
        return;

    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}

// Main function
int main() {
    Node* root = nullptr;
    int choice, key, a, b;
    bool found;

    do {
        cout << "\n--- BST RANGE SEARCHING ---\n";
        cout << "1. Insert Key\n";
        cout << "2. Range Query\n";
        cout << "3. Display BST\n";
        cout << "0. Exit\n";

        cout << "Enter Choice: ";
        cin >> choice;

        switch (choice) {

            case 1:
                cout << "Enter Key: ";
```

main.cpp

Clear Run ▶

```
                cout << "Points in range: ";

                rangeSearch(root, a, b, found);

                if (!found)
                    cout << "No points found";

                cout << endl;
                break;

            case 3:
                cout << "BST Inorder: ";
                inorder(root);
                cout << endl;
                break;

            case 0:
                cout << "Program Exited.\n";
                break;

            default:
                cout << "Invalid Choice!\n";
        }

    } while (choice != 0);

    return 0;
}
```

Result with Various Test Cases:

```
--- BST RANGE SEARCHING ---
1. Insert Key
2. Range Query
3. Display BST
0. Exit
Enter Choice: 1
Enter Key: 20
Key Inserted Successfully

--- BST RANGE SEARCHING ---
1. Insert Key
2. Range Query
3. Display BST
0. Exit
Enter Choice: 1
Enter Key: 10
Key Inserted Successfully

--- BST RANGE SEARCHING ---
1. Insert Key
2. Range Query
3. Display BST
0. Exit
Enter Choice: 1
Enter Key: 30
Key Inserted Successfully
```

```
--- BST RANGE SEARCHING ---
1. Insert Key
2. Range Query
3. Display BST
0. Exit
Enter Choice: 2
Enter Range (a b): 12 28
Points in range: 20

--- BST RANGE SEARCHING ---
1. Insert Key
2. Range Query
3. Display BST
0. Exit
Enter Choice: 3
BST Inorder: 10 20 30
```

Time Complexity and Discussion :

Time complexity describes the amount of time required by an algorithm to execute as the input size increases.

In this problem, the hash table uses Linear Probing to perform insertion, searching and display operations.

Let n represent the number of positions in the hash table.

Time Complexity Table

Operation	Average Case	Worst Case
Insertion	O(log n)	O(n)
Range Searching	O(log n + K)	O(n)
Inorder Display	O(n)	O(n)

Explanation:

- Insertion: The algorithm follows the BST structure to find the correct position.
- Range Searching: It avoids unnecessary subtree traversal and prints only matching keys.
- Inorder Display: Every node is visited exactly once.

Advantages:

- Maintains sorted data.
- Efficient range searching in a balanced tree.
- Avoids unnecessary comparisons.
- Simple implementation

Conclusion:

The 1D Range Searching using Binary Search Tree was successfully implemented using C++.

The program performs insertion and range query operations. It efficiently identifies all keys within a given range while maintaining the sorted property of the Binary Search Tree.

Thus, BST provides an effective approach for one-dimensional range searching.

PROBLEM STATEMENT 4

Unit 6

Implement a Bloom Filter to check wheather a username is likely already taken or definitely available. Use three hash functions and a bit array of size 20.

Operations to be implemented:

1. Add Username
2. Check Username

Imporatant Note:

- False positive are possible.
- False negatives are not possible

Theory :

Algorithm :

Flowchart:

Solution in C++

Discussion on Solution:

The program implements a Bloom Filter using a bit array of size 20 and three hash functions.

Initially, all positions in the bit array contain zero.

When a username is added, three hash functions calculate positions in the bit array. These positions are updated to 1.

When checking a username, the same hash functions calculate the positions again.

If any calculated bit is 0, the username is definitely available.

If all calculated bits are 1, the username is probably taken.

The Bloom Filter provides memory-efficient membership checking without storing complete username records.

Screen Shots of Program :

main.cpp Clear Run ▶

```cpp
#include <iostream>
#include <string>
using namespace std;

#define SIZE 20

// Bloom Filter bit array
int bitArray[SIZE] = {0};

// First hash function
int hash1(string username) {
    unsigned long long sum = 0;

    for (char ch : username)
        sum += (unsigned char)ch;

    return sum % SIZE;
}

// Second hash function
int hash2(string username) {
    unsigned long long sum = 0;

    for (int i = 0; i < username.length(); i++) {
        sum += (unsigned long long)(i + 1) *
               (unsigned char)username[i];
    }
```

main.cpp Clear Run ▶

```cpp
int hash3(string username) {
    unsigned long long hash = 0;

    for (char ch : username) {
        hash = (hash * 31 + (unsigned char)ch) % SIZE;
    }

    return hash;
}

// Add username
void addUsername(string username) {
    int h1 = hash1(username);
    int h2 = hash2(username);
    int h3 = hash3(username);

    bitArray[h1] = 1;
    bitArray[h2] = 1;
    bitArray[h3] = 1;

    cout << "Username Added Successfully\n";
}

// Check username
void checkUsername(string username) {
    int h1 = hash1(username);
    int h2 = hash2(username);
    int h3 = hash3(username);
```

main.cpp

Clear Run ▶

```
    if (bitArray[h1] == 1 &&
        bitArray[h2] == 1 &&
        bitArray[h3] == 1) {

        cout << "Username is Probably Taken\n";
    }
    else {
        cout << "Username is Available\n";
    }
}

// Display bit array
void displayFilter() {
    cout << "\nBloom Filter Bit Array:\n";

    for (int i = 0; i < SIZE; i++) {
        cout << bitArray[i] << " ";
    }

    cout << endl;
}

// Main function
int main() {
    int choice;
    string username;

    do {
        cout << "\n--- BLOOM FILTER ---\n";
```

main.cpp

Clear Run ▶

```
            case 1:
                cout << "Enter Username: ";
                cin >> username;
                addUsername(username);
                break;

            case 2:
                cout << "Enter Username: ";
                cin >> username;
                checkUsername(username);
                break;

            case 3:
                displayFilter();
                break;

            case 0:
                cout << "Program Exited\n";
                break;

            default:
                cout << "Invalid Choice!\n";
        }

    } while (choice != 0);

    return 0;
}
```

Result with Various Test Cases:

Output

```
--- BLOOM FILTER ---
1. Add Username
2. Check Username
3. Display Bit Array
0. Exit
Enter Choice:
1
Enter Username: tusharbisane
Username Added Successfully

--- BLOOM FILTER ---
1. Add Username
2. Check Username
3. Display Bit Array
0. Exit
Enter Choice: 2
Enter Username: teena
Username is Available

--- BLOOM FILTER ---
1. Add Username
2. Check Username
3. Display Bit Array
0. Exit
Enter Choice: 3

Bloom Filter Bit Array:
0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 0 1
```

Time Complexity and Discussion :

Time complexity describes the amount of time required by an algorithm to execute as the input size increases.

In this problem, the hash table uses Linear Probing to perform insertion, searching and display operations.

Let n represent the number of positions in the hash table.

Time Complexity Table

Operation	Average Case	Worst Case
Add Username	O(k * L)	O(n)
Check Username	O(k * L)	O(n)
Display Bit array	O(n)	O(n)

Explanation:

Here, L represents the length of the username. Since there are three hash functions, the operations take O(L) time for a username of length L.

Space Complexity: O(m), where m = 20 bits (excluding temporary input storage).

Advantages:

- Requires very little memory.
- Fast insertion and membership checking.
- No need to store complete usernames.
- Suitable for large datasets.

Conclusion:

The Bloom Filter for Username Availability was successfully implemented using C++.

The program uses a bit array of size 20 and three hash functions to check username availability.

It efficiently performs insertion and membership checking with low memory consumption. False positives are possible, but false negatives are not possible.

Thus, Bloom Filters provide an efficient probabilistic technique for checking the presence of elements in a dataset.